

MAY 2026



MACHINE LEARNING CLASSIFICATION USING TITANIC DATASET

**DEPARTMENT OF COMPUTER AND DATA SCIENCE
NSBM GREEN UNIVERSITY**

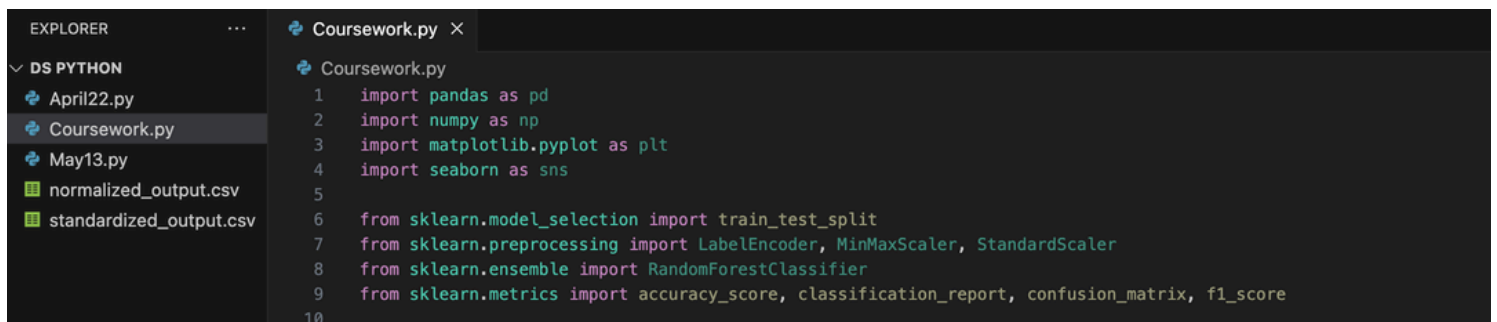
**Prepared by:
M.P. Ranasinghe
38239**

Introduction

- Machine learning is a branch of artificial intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed.
- One important type of machine learning is classification, which is used to predict categorical outcomes based on input data.
- This project focuses on a classification task using the Titanic dataset.
- The Titanic dataset is a publicly available dataset that contains information about passengers who traveled on the Titanic ship, including features such as age, gender, passenger class, fare, and survival status.
- The main objective of this project is to build machine learning models that can predict whether a passenger survived or not based on different passenger characteristics.
- Exploratory Data Analysis (EDA) techniques were used to understand the dataset, identify patterns, detect missing values and outliers, and visualize relationships between variables.
- Machine learning algorithms such as K-Nearest Neighbors (KNN) and Random Forest were implemented and evaluated using performance metrics including accuracy score and F1-score.

Import Libraries

The required Python libraries were imported for data analysis, data preprocessing, visualization, machine learning model building, and evaluation.

A screenshot of a code editor with a dark theme. On the left, the 'EXPLORER' sidebar shows a file tree with 'DS PYTHON' expanded, containing files like 'April22.py', 'Coursework.py', 'May13.py', and two CSV files. The main editor window shows 'Coursework.py' with the following code:

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
8 from sklearn.ensemble import RandomForestClassifier
9 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
10
```

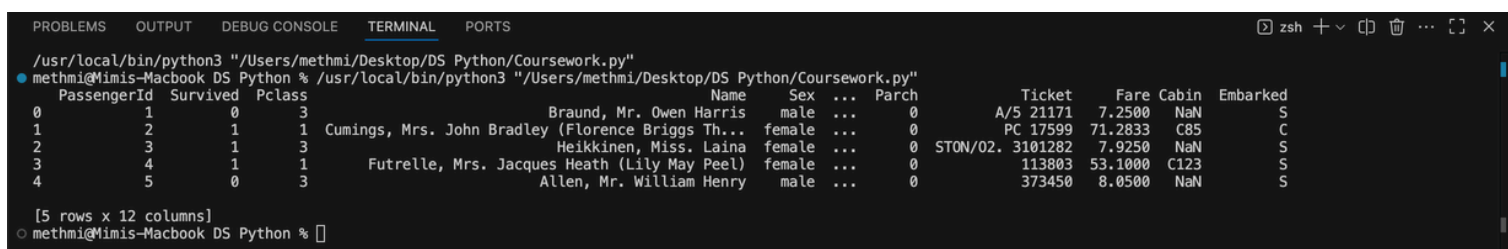
Load Dataset

The dataset contains passenger information from the Titanic ship disaster. Such as age, gender, passenger class, fare and survival status.

A screenshot of a code editor showing the following code:

```
11
12 # Load dataset
13 df = pd.read_csv("/Users/methmi/Desktop/Titanic-Dataset.csv")
14
15 print(df.head())
16
```

Output:

A screenshot of a terminal window showing the output of the code. The output is a table with 12 columns: PassengerId, Survived, Pclass, Name, Sex, Parch, Ticket, Fare, Cabin, Embarked. The first 5 rows are displayed:

PassengerId	Survived	Pclass	Name	Sex	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	Braund, Mr. Owen Harris	male	0	A/5 21171	7.2500	NaN	S
1	2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	0	PC 17599	71.2833	C85	C
2	3	1	Heikkinen, Miss. Laina	female	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	0	113803	53.1000	C123	S
4	5	0	Allen, Mr. William Henry	male	0	373450	8.0500	NaN	S

Data Exploration

The dataset was explored to understand:

- Number of rows and columns
- Data types
- Missing values
- Statistical summaries

```
16
17     print(df.shape)
18
19     print("Data Info:")
20     print(df.info())
21
22     print("Statistical Summary:")
23     print(df.describe())
24
```

Output:

```
(891, 12)
Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   PassengerId      891 non-null    int64
1   Survived         891 non-null    int64
2   Pclass           891 non-null    int64
3   Name             891 non-null    object
4   Sex              891 non-null    object
5   Age              714 non-null    float64
6   SibSp            891 non-null    int64
7   Parch            891 non-null    int64
8   Ticket           891 non-null    object
9   Fare             891 non-null    float64
10  Cabin            204 non-null    object
11  Embarked         889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
Statistical Summary:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
o methmic@Mimis-Macbook DS Python %
```

Missing Values

The `isnull().sum()` function was used to identify the number of missing values in each column of the dataset.

```
25
26     print("Missing Values:")
27     print(df.isnull().sum())
28
```

Output:

```
Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
○ methmi@Mimis-Macbook DS Python %
```

Handle missing values

Missing values were handled before training the models.

- Age values were filled using median.
- Embarked values were filled using mode.
- Cabin column was removed.

```
36
37 # Handle missing values
38 df['Age'].fillna(df['Age'].median(), inplace=True)
39 df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
40
41 # Drop Cabin column
42 df.drop('Cabin', axis=1, inplace=True)
43
44 print("Missing Values After Handling:")
45 print(df.isnull().sum())
46
```

Output:

```
/Users/methmi/Desktop/DS Python/Coursework.py:38: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df['Age'].fillna(df['Age'].median(), inplace=True)
/Users/methmi/Desktop/DS Python/Coursework.py:39: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
Missing Values After Handling:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
```

Duplicate Removal

Duplicate rows were identified and removed to improve data quality.

```
29
30     print("Duplicate Rows:")
31     print(df.duplicated().sum())
32
33     #Remove duplicates
34     df = df.drop_duplicates()
35
```

Output:

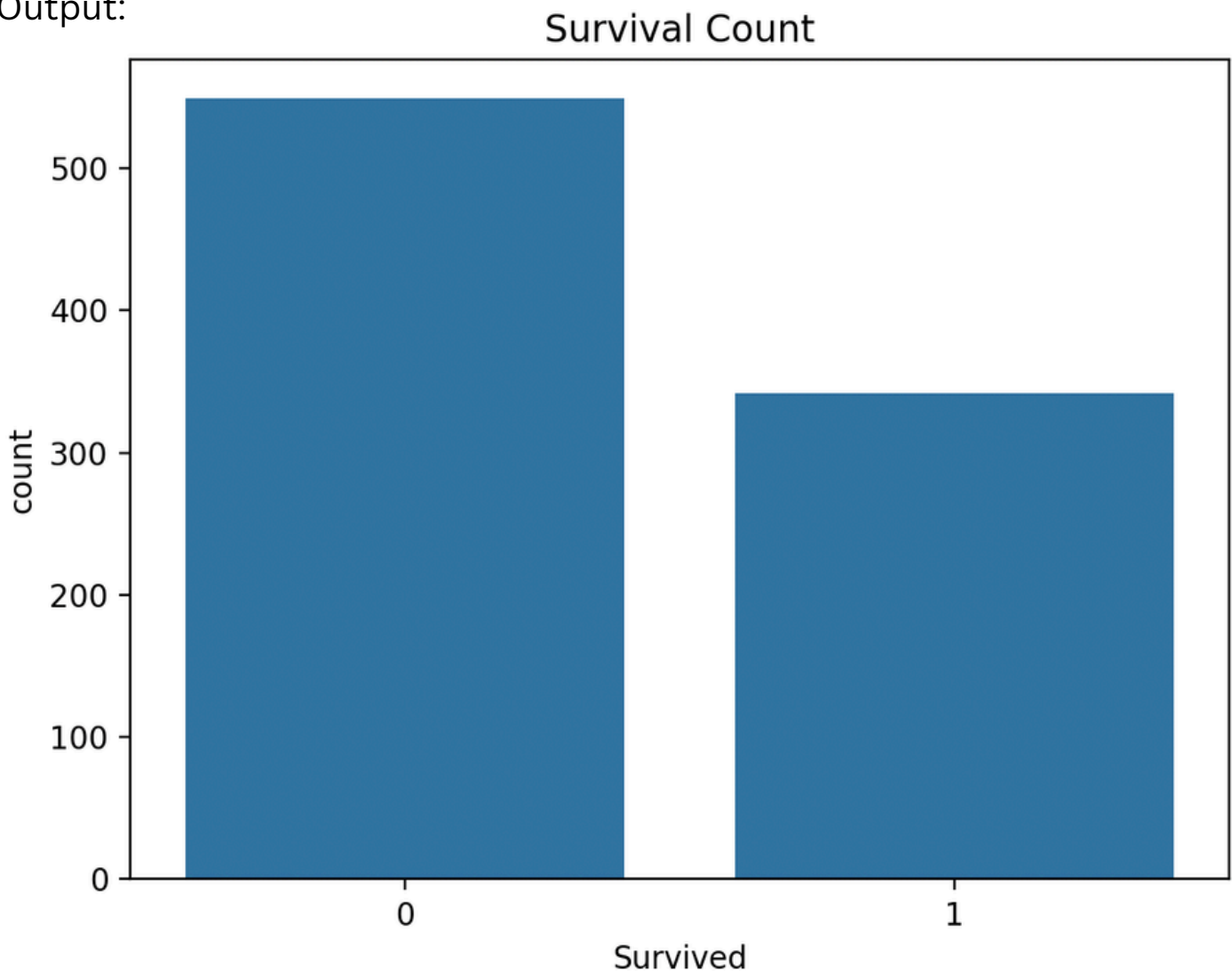
```
Duplicate Rows:
0
methmi@Mimis-Macbook DS Python %
```

Survival Count Visualization

The count plot was used to visualize survival distribution in the dataset.

```
48     # Visualizations
49
50     sns.countplot(x='Survived', data=df)
51     plt.title("Survival Count")
52     plt.show()
53
```

Output:



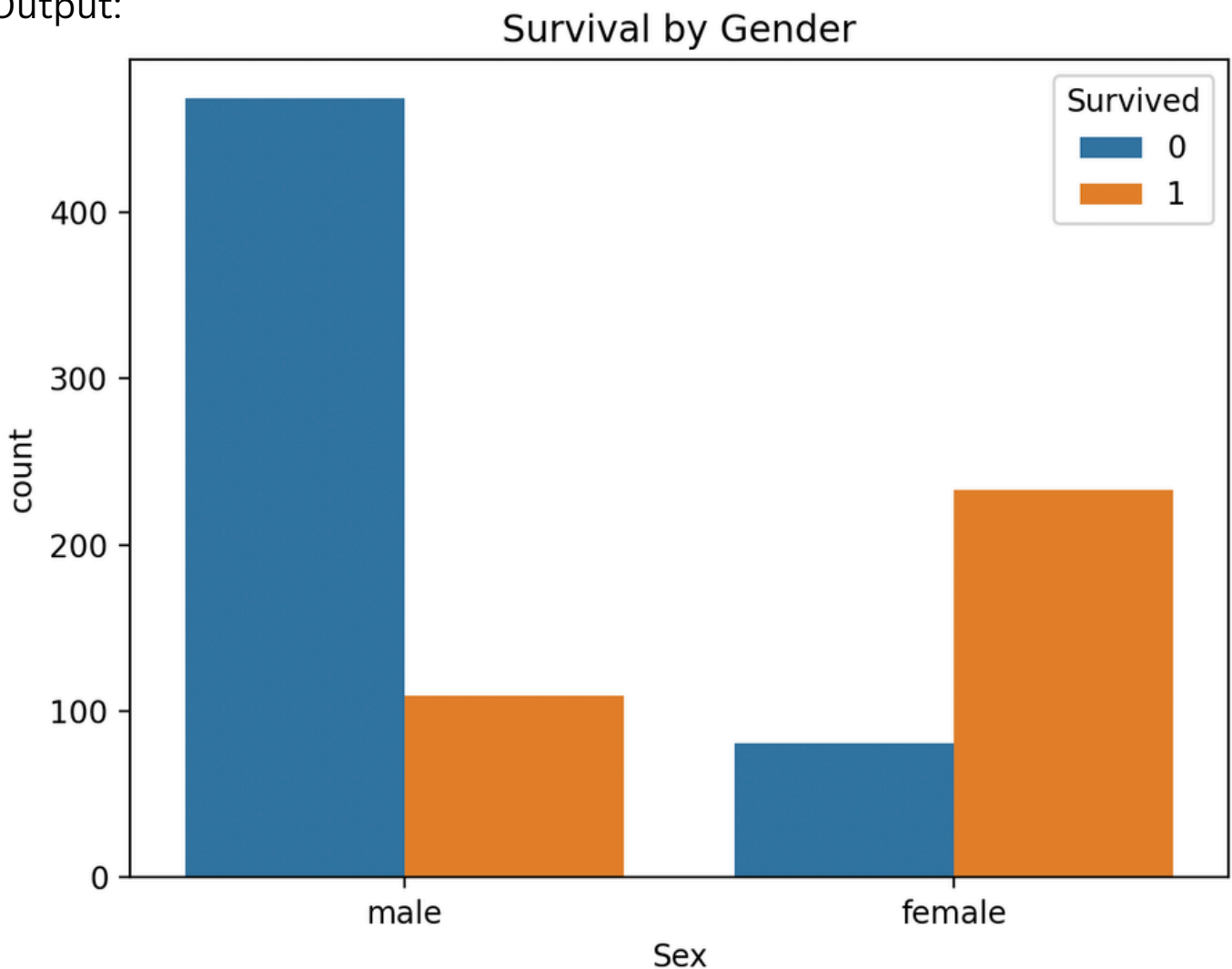
Interpretation: More passengers died than survived.

Survival by Gender

This graph was used to compare survival rates between male and female passengers.

```
54
55     sns.countplot(x='Sex', hue='Survived', data=df)
56     plt.title("Survival by Gender")
57     plt.show()
58
```

Output:



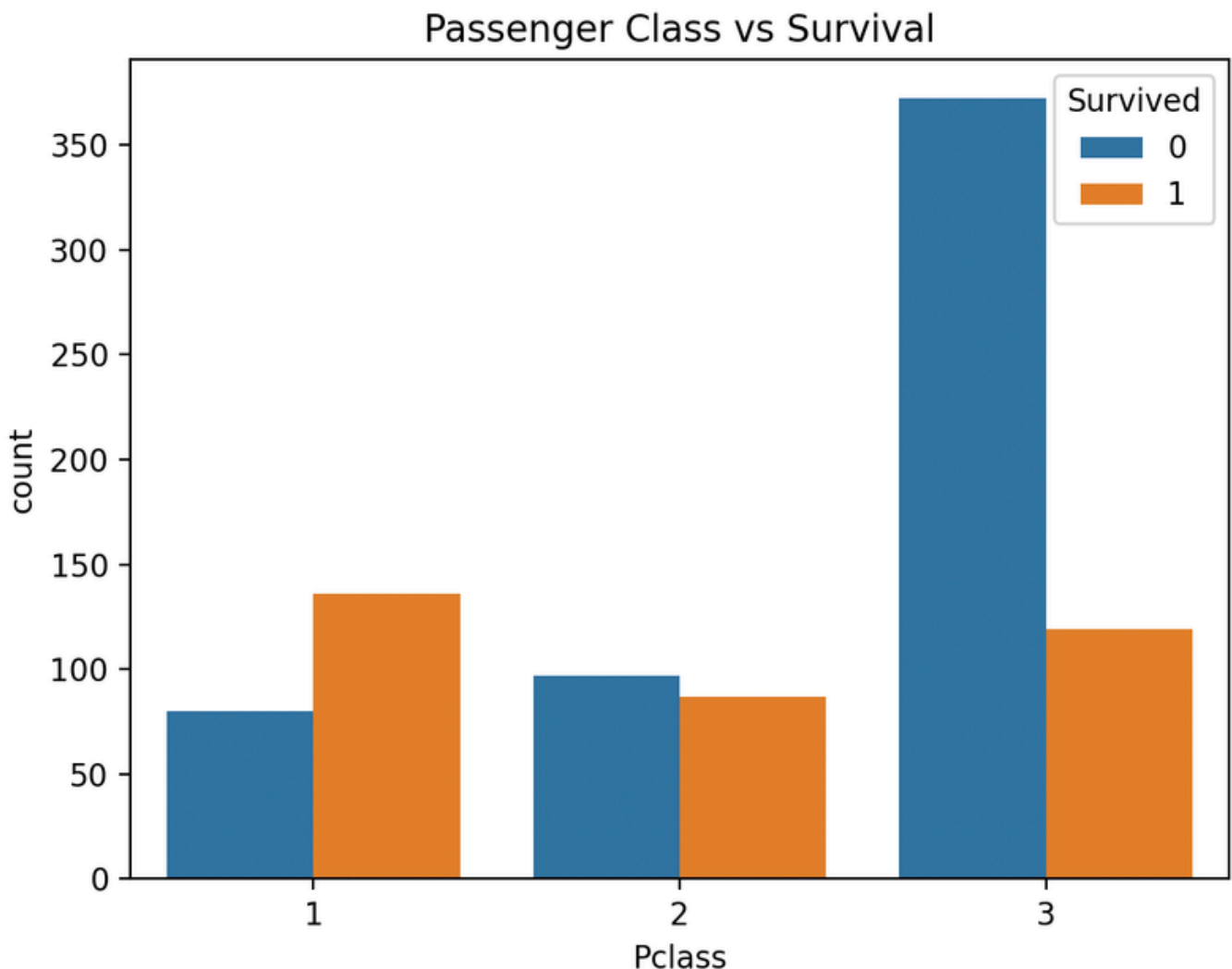
Interpretation: Female passengers had higher survival rates than male passengers.

Passenger Class vs Survival

This visualization shows the relationship between passenger class and survival.

```
59
60 sns.countplot(x='Pclass', hue='Survived', data=df)
61 plt.title("Passenger Class vs Survival")
62 plt.show()
63
```

Output:



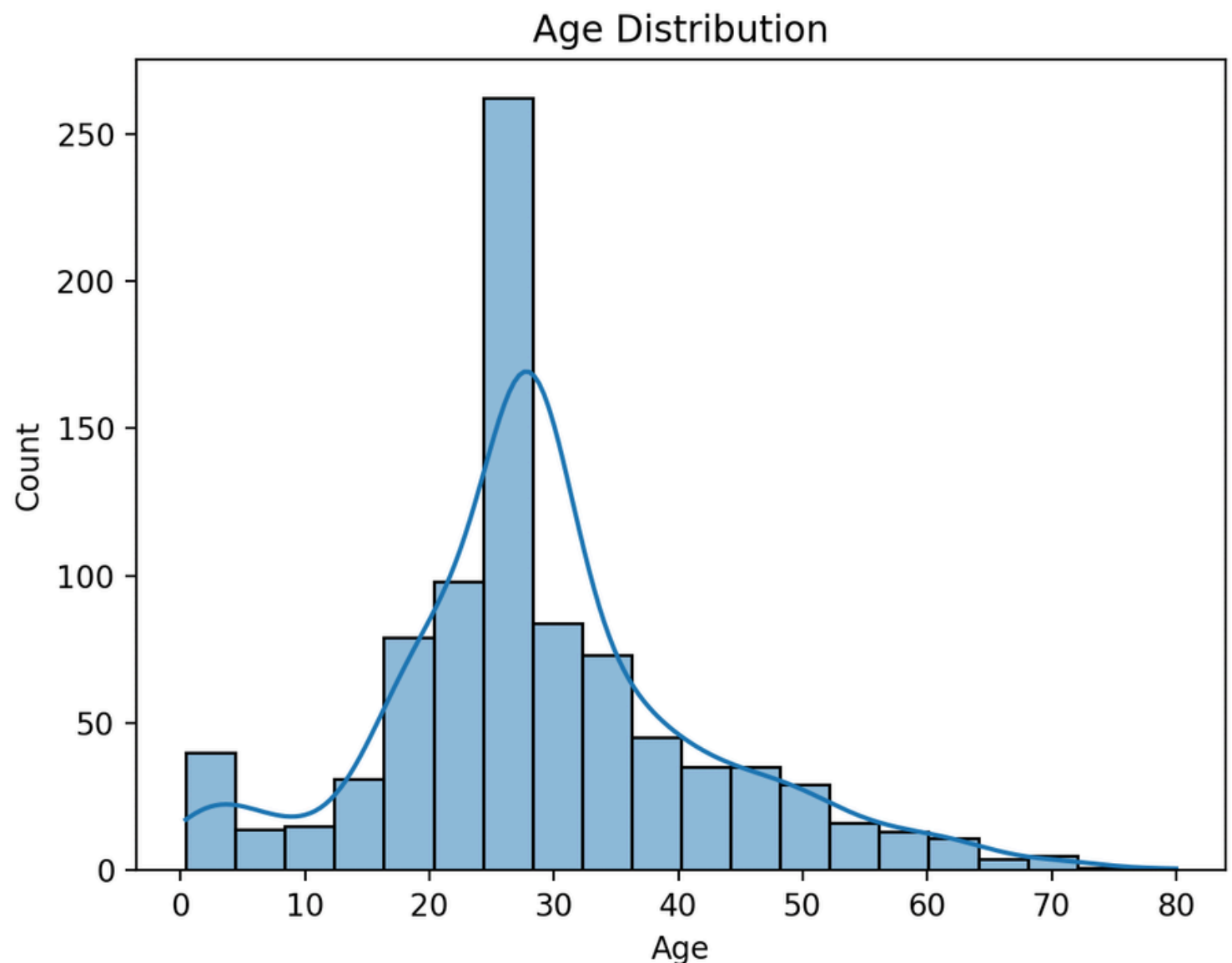
Interpretation: First-class passengers had better survival rates.

Age Distribution

The histogram was used to analyze passenger age distribution.

```
64
65 sns.histplot(df['Age'], bins=20, kde=True)
66 plt.title("Age Distribution")
67 plt.show()
68
```

Output:

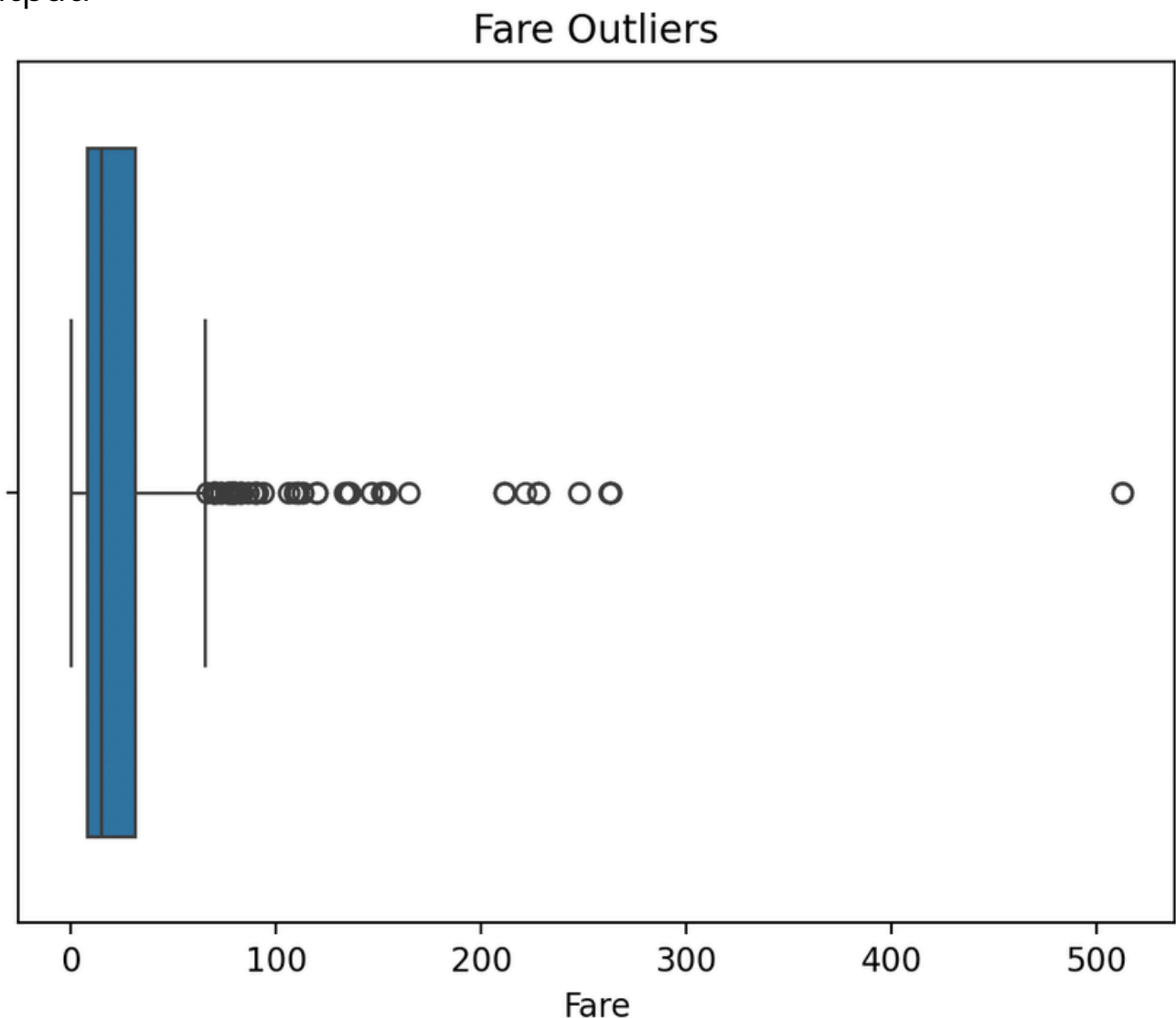


Fare Outliers

The boxplot was used to identify outliers in the Fare column.

```
69
70 sns.boxplot(x=df['Fare'])
71 plt.title("Fare Outliers")
72 plt.show()
73
```

Output:



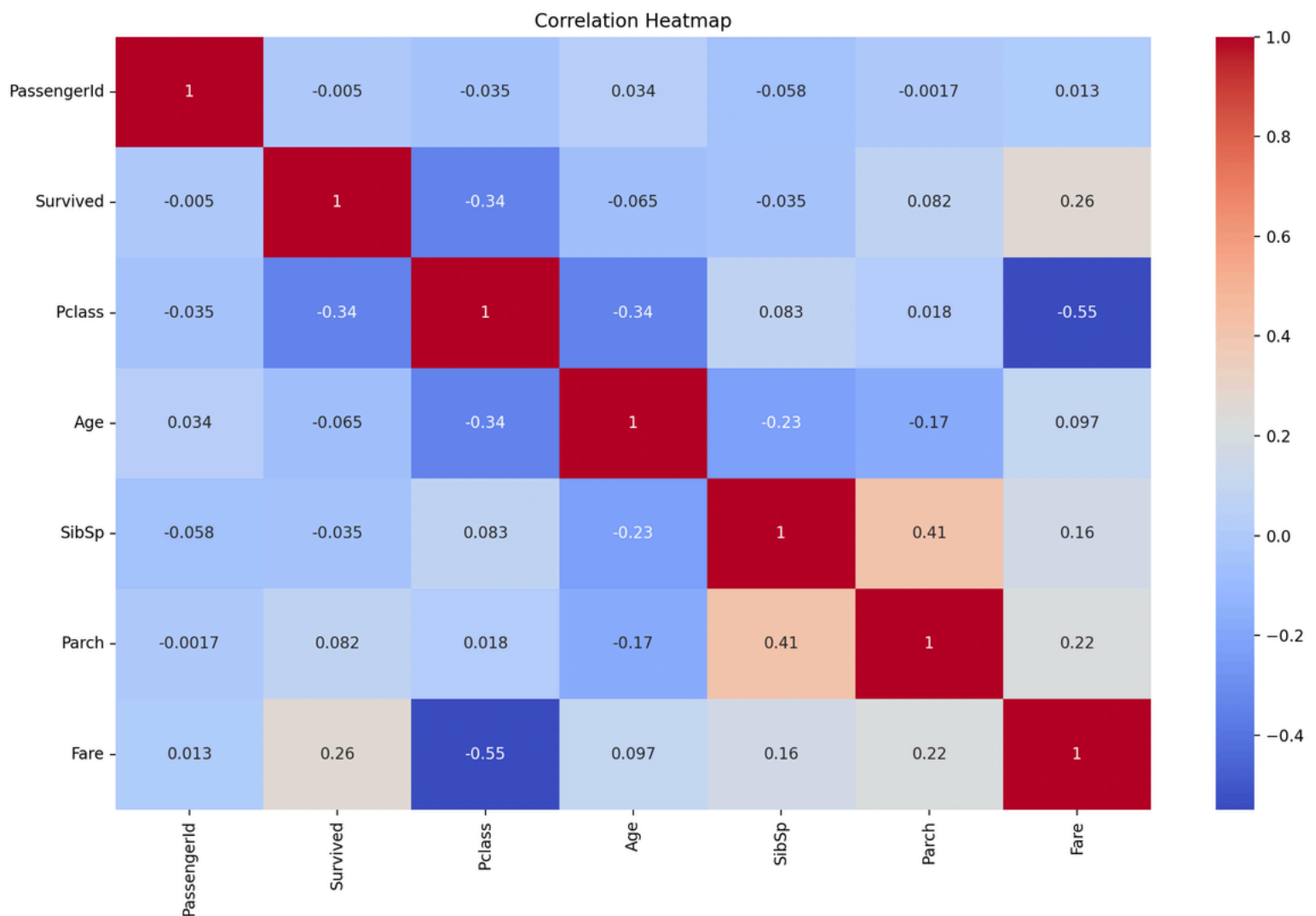
Interpretation: Some passengers paid very high ticket fares compared to others.

Correlation Heatmap

A heatmap was used to visualize the correlation between numerical variables in the dataset.

```
74
75 sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
76 plt.title("Correlation Heatmap")
77 plt.show()
78
```

Output:



Interpretation: The heatmap showed positive and negative relationships between variables and survival.

Feature Selection

Unnecessary columns were removed because they do not significantly affect prediction.

```
80
81 # Feature selection
82 df_features = df.drop(columns=["PassengerId", "Name", "Ticket"], errors="ignore")
83
```

Feature Encoding

Categorical values were converted into numerical values using Label Encoding.

```
85 # Encode categorical variables
86 le = LabelEncoder()
87 df['Sex'] = le.fit_transform(df['Sex'])
88 df['Embarked'] = le.fit_transform(df['Embarked'])
89
```

Normalization (Min-Max Scaling)

Normalization was applied using Min-Max scaling to scale values between 0 and 1.

```
89
90 #Normalization (Min-Max Scaling)
91 from sklearn.preprocessing import MinMaxScaler
92
93 scaler = MinMaxScaler()
94
95 df_normalized = pd.DataFrame(
96     scaler.fit_transform(df.select_dtypes(include=np.number)),
97     columns=df.select_dtypes(include=np.number).columns
98 )
99
100 print("Normalized Data:")
101 print(df_normalized.head())
102
```

Output:

Normalized Data:

	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	0.000000	0.0	1.0	1.0	0.271174	0.125	0.0	0.014151	1.0
1	0.001124	1.0	0.0	0.0	0.472229	0.125	0.0	0.139136	0.0
2	0.002247	1.0	1.0	0.0	0.321438	0.000	0.0	0.015469	1.0
3	0.003371	1.0	0.0	0.0	0.434531	0.125	0.0	0.103644	1.0
4	0.004494	0.0	1.0	1.0	0.434531	0.000	0.0	0.015713	1.0

Standardization (Z-score Scaling)

Standardization was applied to transform data so that it has mean 0 and standard deviation 1.

```
103 #Standardization (Z-score Scaling)
104
105 from sklearn.preprocessing import StandardScaler
106
107 # Select only numeric columns
108 df_features_numeric = df_features.select_dtypes(include=np.number)
109
110 standard = StandardScaler()
111
112 df_standardized = pd.DataFrame(
113     standard.fit_transform(df_features_numeric),
114     columns=df_features_numeric.columns
115 )
116
117 print("Standardized Data:")
118 print(df_standardized.head())
119
```

Output:

```
Standardized Data:
   Survived  Pclass    Age  SibSp  Parch    Fare
0 -0.789272  0.827377 -0.565736  0.432793 -0.473674 -0.502445
1  1.266990 -1.566107  0.663861  0.432793 -0.473674  0.786845
2  1.266990  0.827377 -0.258337 -0.474545 -0.473674 -0.488854
3  1.266990 -1.566107  0.433312  0.432793 -0.473674  0.420730
4 -0.789272  0.827377  0.433312 -0.474545 -0.473674 -0.486337
○ methmi@Mimis-Macbook DS Python %
```


Outlier Handling Using IQR Method

The IQR method was used to identify and handle outliers in the numerical features. Extreme values outside the acceptable range were replaced with the lower or upper boundary values instead of removing them. This helped reduce the impact of outliers and improved the quality of the dataset for model training.

```
# Check statistics BEFORE outlier handling
print("Before Outlier Handling:")
print(df_features.describe())

# Outlier handling (IQR capping)
def cap_outliers_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[column] = np.where(df[column] < lower, lower, df[column])
    df[column] = np.where(df[column] > upper, upper, df[column])

    return df

numeric_cols = df_features.select_dtypes(include=np.number).columns

for col in numeric_cols:
    df_features = cap_outliers_iqr(df_features, col)

# Check statistics AFTER outlier handling
print("\nAfter Outlier Handling:")
print(df_features.describe())
```

Output:

```
Before Outlier Handling:
count    Survived    Pclass    ...    Parch    Fare
mean     0.383838    2.308642    ...    0.381594    32.204208
std      0.486592    0.836071    ...    0.806057    49.693429
min      0.000000    1.000000    ...    0.000000    0.000000
25%      0.000000    2.000000    ...    0.000000    7.910400
50%      0.000000    3.000000    ...    0.000000    14.454200
75%      1.000000    3.000000    ...    0.000000    31.000000
max      1.000000    3.000000    ...    6.000000    512.329200

[8 rows x 6 columns]

After Outlier Handling:
count    Survived    Pclass    ...    Parch    Fare
mean     0.383838    2.308642    ...    0.0    24.046813
std      0.486592    0.836071    ...    0.0    20.481625
min      0.000000    1.000000    ...    0.0    0.000000
25%      0.000000    2.000000    ...    0.0    7.910400
50%      0.000000    3.000000    ...    0.0    14.454200
75%      1.000000    3.000000    ...    0.0    31.000000
max      1.000000    3.000000    ...    0.0    65.634400

[8 rows x 6 columns]
methmi@Mimis-Macbook DS Python %
```

The output is a cleaned dataset with reduced extreme values. Comparing the summary statistics before and after outlier handling shows that the outliers were successfully controlled.

Splitting the Dataset into Training and Testing Sets

The dataset was divided into features (X) and the target variable (y). The Survived column was selected as the target variable, while all other columns were used as input features.

The `train_test_split()` function was then used to split the dataset into training and testing sets:

- 80% of the data was used for training the model.
- 20% of the data was used for testing the model.
- `random_state=42` ensures that the same split is produced each time the code is run.

```
# Split data
X = df_features.drop("Survived", axis=1)
y = df_features["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

Output:

```
X_train shape: (712, 7)
X_test shape: (179, 7)
y_train shape: (712,)
y_test shape: (179,)
```

This code creates four datasets:

- X_train – Training features
- X_test – Testing features
- y_train – Training target values
- y_test – Testing target values

These datasets are then used to train and evaluate the machine learning model.

Model

This code is used to build a Random Forest model to solve a classification problem. Its purpose is to learn patterns from the training data (X_train, y_train) and then use that learned model to predict the class labels of new data (X_test). The final predictions are stored in y_pred, which is later used to evaluate how well the model performs.

```
# Model
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
print(y_pred)
```

Output:

```
[0. 0. 0. 1. 0. 1. 1. 0. 1. 1. 1. 0. 0. 0. 0. 1. 1. 1. 0. 0. 0. 1. 0. 0.
 0. 0. 0. 0. 0. 1. 0. 1. 1. 1. 0. 0. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 1. 1. 0. 1. 0. 1. 0. 1. 1. 0. 0. 1. 1. 0. 0. 1. 0. 0. 0. 1. 1. 1.
 1. 1. 0. 0. 1. 1. 1. 0. 0. 1. 1. 0. 0. 0. 1. 1. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 1. 0. 0. 0. 1. 0. 0. 1. 1. 0. 1. 0. 1. 0. 1. 1. 0. 0. 1. 0. 1. 0.
 0. 1. 1. 1. 0. 0. 1. 0. 0. 0. 1. 0. 0. 1. 1. 0. 1. 0. 0. 0. 0. 1. 1. 0.
 0. 1. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0. 0. 1. 1. 1. 0. 0. 0. 1. 0. 0. 0. 1.
 0. 0. 0. 1. 1. 0. 0. 0. 0. 1. 1.]
```

Evaluation

This is used to evaluate how well the Random Forest model is performing by comparing the predicted values (`y_pred`) with the actual values (`y_test`).

It calculates and displays three main metrics:

- Accuracy Score: Shows the overall percentage of correct predictions.
- F1 Score: Measures the balance between precision and recall, especially useful for classification problems.
- Classification Report: Gives a detailed summary of performance including precision, recall, and F1-score for each class.

```
# Evaluation
print("Accuracy Score:")
print(accuracy_score(y_test, y_pred))

print("F1 Score:")
print(f1_score(y_test, y_pred))

print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
Accuracy Score:
0.8324022346368715
F1 Score:
0.7916666666666666
Classification Report:
      precision    recall  f1-score   support

    0.0         0.84      0.88      0.86         105
    1.0         0.81      0.77      0.79          74

 accuracy          0.83          179
  macro avg         0.83          179
 weighted avg         0.83          179
```

Confusion Matrix

This code is used to visualize and analyze the performance of the classification model using a confusion matrix.

It first calculates the confusion matrix by comparing actual values (`y_test`) with predicted values (`y_pred`). Then, it uses a heatmap to display the results in a clear visual format, showing how many predictions were correct and incorrect for each class. Finally, it prints a confirmation message that the classification process is completed.

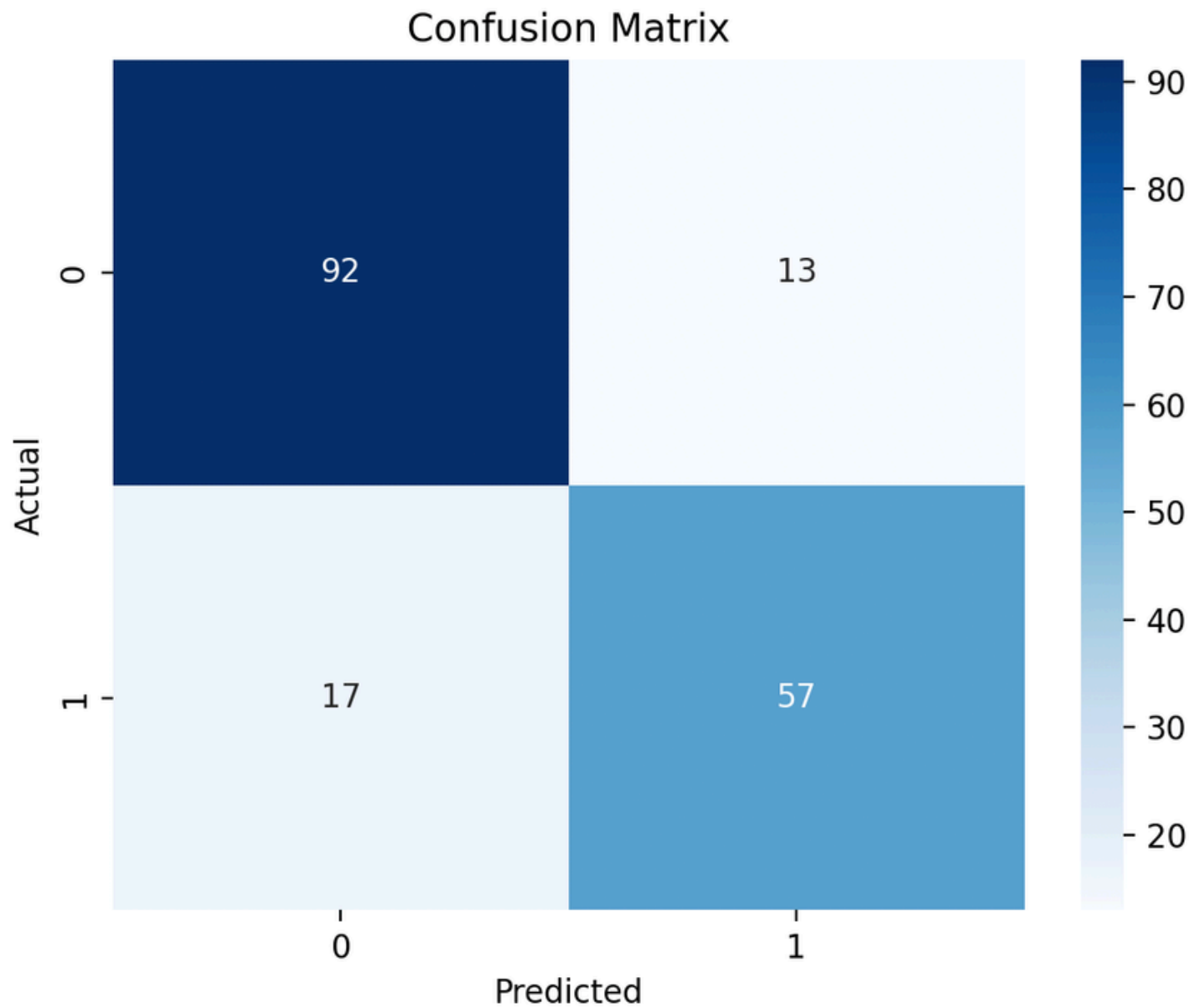
```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print("Classification Completed Successfully!")
```


Output:

```
Classification Completed Successfully!  
methmi@Mimis-Macbook DS Python %
```



Conclusion

In this coursework, the Titanic dataset was used to perform a classification task to predict passenger survival. Exploratory Data Analysis (EDA) was carried out to understand the dataset, identify missing values, duplicates, outliers, and relationships between variables. Data preprocessing techniques such as handling missing values, encoding categorical variables, scaling, and outlier treatment were applied to improve data quality.

A Random Forest Classifier was then developed and trained using the processed dataset. The model achieved an accuracy score of 83.24% and an F1-score of 79.17%, indicating good predictive performance. The classification report and confusion matrix further demonstrated that the model was able to classify both survival and non-survival cases effectively.

Overall, this project successfully demonstrated the complete machine learning workflow, including data exploration, preprocessing, model development, evaluation, and interpretation of results. The findings suggest that factors such as passenger class, gender, and age had a significant influence on survival outcomes. The Random Forest algorithm proved to be an effective method for this classification task.

End of the report